

**REAL-TIME DRIVER DROWSINESS
DETECTION SYSTEM USING EYE
ASPECT RATIO AND LIGHTWEIGHT
MACHINE LEARNING**

AI23331 - FUNDAMENTAL OF MACHINE LEARNING

Submitted by

KRITHIKA P V 230701157

KAVEESHAA S 230701146

in partial fulfillment for the award of the degree

of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING



RAJALAKSHMI ENGINEERING COLLEGE

ANNA UNIVERSITY, CHENNAI

MAY 2025

RAJALAKSHMI ENGINEERING COLLEGE, CHENNAI

BONAFIDE CERTIFICATE

Certified that this Project titled **“REAL-TIME DRIVER DROWSINESS DETECTION SYSTEM USING EYEASPECT RATIO AND LIGHTWEIGHT MACHINE LEARNING** “is the bonafide work of **“KRITHIKA P V (230701157) AND KAVEESHAA S (230701146)”** who carried

out the work under my supervision. Certified further that to the best of my knowledge the work reported herein does not form part of any other thesis or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

SIGNATURE

Dr. P. Kumar., M.E., Ph.D.,

HEAD OF THE DEPARTMENT

Professor

Department of Computer Science

and Engineering,

Rajalakshmi Engineering College,

Chennai - 602 105.

SIGNATURE

Dr. M. Rakesh Kumar., M.E., Ph.D.,

SUPERVISOR

Assistant Professor

Department of Computer Science

and Engineering,

Rajalakshmi Engineering

College, Chennai-602 105.

Submitted to Mini Project Viva-Voce Examination held on _____

Internal Examiner

External Examiner

ABSTRACT

Driver Drowsiness Detection System is an intelligent web-based application developed to analyse and monitor users' digital behaviour, with the goal of identifying patterns that may indicate social media overuse or digital addiction. The system continuously observes key usage metrics such as daily screen time, frequency of device unlocks, time spent on social media applications, and late-night usage patterns. Using these inputs, it applies a rule-based simulation model to generate a Digital Addiction Risk Score ranging from low to high.

Based on the calculated score, the application classifies user behaviour into categories such as normal, moderate, or high risk. It provides real-time insights through visual dashboards, including charts and dynamic statistics, allowing users to better understand their usage habits. The system also generates personalized recommendations, such as reducing screen time, taking breaks, or enabling focus sessions, to promote healthier digital routines.

A key feature of **Driver Drowsiness Detection System** is its interactive chatbot, which responds to user queries, provides usage summaries, and suggests actionable steps to improve digital well-being. Additionally, the application includes a real-time alert system that notifies users when excessive usage or unhealthy patterns, such as prolonged screen time or late-night activity, are detected.

Although the system uses simulated logic instead of a fully integrated machine learning model, it effectively mimics AI-driven behaviour, making it suitable for demonstration and practical understanding. Overall, **MindTrack AI** aims to enhance user awareness, improve productivity, and encourage balanced technology usage through an engaging and user-friendly interface.

ACKNOWLEDGMENT

Initially we thank the Almighty for being with us through every walk of our life and showering his blessings through the endeavor to put forth this report. Our sincere thanks to our Chairman **Mr. S. MEGANATHAN, B.E, F.I.E.**, our Vice Chairman **Mr. ABHAY SHANKAR MEGANATHAN, B.E., M.S.**, and our respected Chairperson **Dr. (Mrs.) THANGAM MEGANATHAN, Ph.D.**, for providing us with the requisite infrastructure and sincere endeavoring in educating us in their premier institution.

Our sincere thanks to **Dr. S.N. MURUGESAN, M.E., Ph.D.**, our beloved Principal for his kind support and facilities provided to complete our work in time. We express our sincere thanks to **Dr. P. KUMAR, M.E., Ph.D.**, Professor and Head of the Department of Computer Science and Engineering for his guidance and encouragement throughout the project work. We convey our sincere and deepest gratitude to our internal guides **Dr. JINU SHOPIA.** and **Dr. M. RAKESH KUMAR**, We are very glad to thank our Project Coordinator, **Dr. M. RAKESH KUMAR** Assistant Professor Department of Computer Science and Engineering for his useful tips during our review to build our project.

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	i
	ACKNOWLEDGMENT	iv
	LIST OF TABLES	vii
	LIST OF FIGURES	viii
	LIST OF ABBREVIATIONS	ix
1.	INTRODUCTION	1
	1.1 GENERAL	1
	1.2 OBJECTIVES	2
	1.3 EXISTING SYSTEM	2
2.	LITERATURE SURVEY	3
3.	PROPOSED SYSTEM	14
	3.1 GENERAL	14
	3.2 SYSTEM ARCHITECTURE DIAGRAM	14
	3.3 DEVELOPMENT ENVIRONMENT	15
	3.3.1 HARDWARE REQUIREMENTS	15
	3.3.2 SOFTWARE REQUIREMENTS	16
	3.4 DESIGN THE ENTIRE SYSTEM	16
	3.4.1 ACTIVITYY DIAGRAM	17
	3.4.2 DATA FLOW DIAGRAM	18

	3.5 STATISTICAL ANALYSIS	18
4.	MODULE DESCRIPTION	20
	4.1 SYSTEM ARCHITECTURE	20
	4.1.1 USER INTERFACE DESIGN	20
	4.1.2 BACK END INFRASTRUCTURE	21
	4.2 DATA COLLECTION & PREPROCESSING	21
	4.2.1 DATASET & DATA LABELLING	21
	4.2.2 DATA PREPROCESSING	21
	4.2.3 FEATURE SELECTION	22
	4.2.4 CLASSIFICATION & MODEL SELECTION	22
	4.2.5 PERFORMANCE EVALUATION	23
	4.2.6 MODEL DEPLOYMENT	23
	4.2.7 CENTRALIZED SERVER & DATABASE	23
	4.3 SYSTEM WORKFLOW	23
	4.3.1 USER INTERACTION	23
	4.3.2 FAKE PROFILE DETECTION	24
	4.3.3 BLOCKCHAIN INTEGRATION	24
	4.3.4 FRAUD PREVENTION & REPORTING	24
	4.3.5 CONTINUOUS LEARNING AND IMPROVEMENT	24
5.	IMPLEMENTATIONS AND RESULTS	24

	5.1 IMPLEMENTATION	25
	5.2 OUTPUT SCREENSHOTS	25
6.	CONCLUSION AND FUTURE ENHANCEMENT	30
	6.1 CONCLUSION	30
	6.2 FUTURE ENHANCEMENT	30
	REFERENCES	32
	PUBLICATIONS	36

LIST OF TABLES

TABLE NO	TITLE	PAGE NO
3.1	HARDWARE REQUIREMENTS	13
3.2	SOFTWARE REQUIREMENTS	14
3.3	COMPARISON OF FEATURES	19

LIST OF FIGURES

FIGURE NO	TITLE	PAGE NO
3.1	SYSTEM ARCHITECTURE	7
3.2	ACTIVITY DIAGRAM	9
3.3	DFD DIAGRAM	10
3.4	COMPARISON GRAPH	12
4.1	CORRELATION HEATMAP	18
5.1	DATASET FOR TRAINING	21
5.2	PERFORMANCE EVALUATION AND OPTIMIZATION	21
5.3	CONFUSION MATRIX	22
5.4	BEST ALGORITHM	22
5.5	LINEAR REGRESSION COEFFICIENTS	22
5.6	WEB PAGE	23
5.7	CHATBOT	23

ABSTRACT

Driver drowsiness is one of the major causes of road accidents and transportation safety issues worldwide. Continuous driving, lack of sleep, mental stress, and fatigue reduce the alertness level of drivers and increase the possibility of accidents. Drowsy drivers often experience slower reaction times, poor concentration, and temporary loss of attention, which may lead to dangerous situations on roads. Therefore, early detection of drowsiness is very important for improving driver safety and preventing accidents.

The proposed “Real-Time Driver Drowsiness Detection System Using Eye Aspect Ratio and Lightweight Machine Learning” is developed using Computer Vision and Machine Learning techniques to monitor driver alertness continuously in real time. The system captures webcam video frames and processes them using MediaPipe Face Mesh for facial landmark extraction. Important eye landmark points are identified, and Eye Aspect Ratio (EAR) values are calculated to determine whether the driver’s eyes are open or closed.

The calculated EAR values are processed using a Logistic Regression model that classifies the driver state into Alert or Drowsy categories. If the system detects continuous eye closure or drowsiness for several frames, an audio alert is generated immediately to warn the driver and reduce accident risks. The frontend dashboard displays live webcam feed, EAR values, confidence percentages, detection logs, and alert notifications dynamically in real time.

The application is implemented using Python, Flask, OpenCV, MediaPipe Face Mesh, HTML, CSS, JavaScript, and Web Audio API. The system is lightweight, GPU-free, and capable of running efficiently on standard

hardware systems without requiring expensive sensors or specialized devices. Overall, the proposed system provides an efficient, low-cost, and practical solution for real-time driver monitoring and road safety improvement through intelligent drowsiness detection.

1. INTRODUCTION

CHAPTER 1

Driver drowsiness is one of the major causes of road accidents and traffic-related fatalities across the world. Fatigue, lack of sleep, stress, long driving hours, and continuous concentration reduce the alertness level of drivers and affect their reaction time. When drivers become drowsy, they may lose focus temporarily, respond slowly to road conditions, or even fall asleep while driving. These situations increase the risk of accidents, injuries, and loss of life. Therefore, detecting driver drowsiness at an early stage is essential for improving transportation safety and preventing accidents.

Traditional drowsiness detection systems mainly use physiological sensors such as EEG, EOG, and heart rate monitoring devices to identify fatigue conditions. Although these methods provide accurate results, they require wearable sensors and expensive hardware, making them uncomfortable and difficult for practical everyday use. Vehicle-based monitoring systems analyze steering patterns, lane deviation, and braking behavior, but they do not directly monitor the physical condition of drivers. Modern Deep Learning approaches using CNN and LSTM models improve prediction

accuracy but require large datasets, GPU hardware, and high computational resources.

The proposed “Real-Time Driver Drowsiness Detection System Using Eye Aspect Ratio and Lightweight Machine Learning” provides a lightweight and efficient solution using Computer Vision and Machine Learning techniques. The system continuously captures webcam video frames and processes them using MediaPipe Face Mesh to extract facial landmark points. Eye coordinates are identified from the facial landmarks, and Eye Aspect Ratio (EAR) values are calculated to determine eye openness and closure patterns associated with drowsiness.

The calculated EAR values are processed using a Logistic Regression model that classifies the driver state into Alert or Drowsy categories in real time. If the system detects continuous eye closure for multiple frames, it generates an audio alert to warn the driver immediately. The application includes an interactive frontend dashboard that displays live webcam feed, EAR values, confidence percentages, system status, and detection logs dynamically.

The proposed system is implemented using Python, Flask, OpenCV, MediaPipe Face Mesh, HTML, CSS, JavaScript, and Web Audio API. The lightweight architecture ensures low computational complexity and allows deployment on standard CPU-based systems without requiring expensive hardware or GPUs. Overall, the system aims to improve road safety by providing an efficient, real-time, and intelligent driver monitoring solution capable of detecting drowsiness accurately and generating timely warning alerts.

Driver fatigue and drowsiness are major causes of transportation accidents across the world. Long driving hours, lack of sleep, stress, and reduced concentration affect driver alertness and increase the possibility of accidents. The proposed Driver Drowsiness Detection System continuously monitors eye movements and facial behavior using Computer Vision and Machine Learning techniques. Webcam frames are processed using MediaPipe Face Mesh, and Eye Aspect Ratio values are calculated to identify drowsiness patterns. Logistic Regression predicts whether the driver is alert or drowsy in real time.

1.1 OBJECTIVES

To develop a real-time driver monitoring system

The primary objective of the project is to develop a real-time monitoring system that continuously observes the driver's facial and eye movements during driving. The system processes live video frames using Computer Vision techniques to analyze alertness conditions without causing any disturbance to the driver.

To detect facial landmarks using MediaPipe Face Mesh

The system uses MediaPipe Face Mesh to identify 468 facial landmark points from the driver's face. These landmarks help locate the eye regions accurately and improve the precision of eye movement analysis during real-time monitoring.

To calculate Eye Aspect Ratio (EAR)

The project aims to calculate the Eye Aspect Ratio using selected eye landmark coordinates. EAR is used to measure eye openness and detect

blinking or prolonged eye closure. This value acts as the primary feature for drowsiness detection.

To classify driver state using Machine Learning

The system uses Logistic Regression to classify the driver's condition into Alert or Drowsy categories based on EAR values. The model performs fast and efficient real-time classification with minimal computational requirements.

To generate warning alerts during drowsiness

Whenever the system detects continuous eye closure or drowsiness patterns, it activates an audio alert to notify the driver immediately. This alert mechanism helps prevent accidents caused by fatigue and lack of attention.

To provide an interactive web dashboard

The project also aims to provide a user-friendly dashboard interface that displays live webcam feed, EAR values, prediction confidence, and system status. This improves usability and helps users understand the monitoring process clearly.

To reduce accidents caused by fatigue

The overall objective of the system is to improve road safety and reduce accidents caused by drowsy driving through continuous real-time monitoring and early warning mechanisms.

.

1.2 EXISTING SYSTEM

Existing drowsiness detection systems mainly use physiological methods, vehicle-based monitoring systems, and vision-based techniques. Physiological systems use sensors such as EEG, EOG, and heart rate monitors to detect fatigue by analyzing brain activity and eye movement signals. Although these systems provide highly accurate results, they require wearable devices and specialized hardware equipment. This makes them uncomfortable, expensive, and impractical for daily use.

Vehicle-based monitoring systems analyze driving behavior such as steering movement, lane deviation, braking patterns, and vehicle speed variations to detect drowsiness. These methods do not directly monitor the driver's physical condition and may fail to provide early warnings before dangerous situations occur. They also depend heavily on vehicle-specific sensors and environmental conditions.

Many modern vision-based systems use Deep Learning models such as CNNs and LSTMs for facial analysis and eye tracking. These systems require large datasets, powerful GPUs, and high computational resources for training and deployment. Real-time performance becomes difficult on normal systems due to increased processing complexity.

Some existing systems also suffer from poor performance under low-light conditions, face occlusion, and rapid head movements. Most systems lack lightweight implementation and cost-effective deployment for real-world applications.

The proposed Driver Drowsiness Detection System overcomes these limitations by using MediaPipe Face Mesh and Logistic Regression to

provide lightweight, real-time, and accurate detection using only a webcam and standard hardware.

.

2. LITERATURE SURVEY

Several studies have explored drowsiness detection using Computer Vision techniques. Soukupová and Čech introduced the Eye Aspect Ratio method for eye blink detection. MediaPipe Face Mesh improved real-time facial landmark extraction using lightweight processing. Deep Learning methods such as CNN and LSTM have also been used for drowsiness monitoring, but these approaches require GPUs and large datasets. The proposed system focuses on lightweight real-time implementation using EAR and Logistic Regression.

3. PROPOSED SYSTEM

Driver drowsiness detection has become an important research area in the fields of Computer Vision, Machine Learning, and Intelligent Transportation Systems. Several researchers have proposed different approaches for monitoring driver alertness and detecting fatigue conditions using physiological signals, vehicle behavior analysis, and vision-based monitoring techniques. These systems aim to reduce accidents caused by drowsy driving and improve transportation safety.

Soukupová and Čech introduced a real-time eye blink detection method using facial landmarks and Eye Aspect Ratio (EAR). Their research demonstrated that EAR is an efficient and lightweight method for detecting eye closure and blinking patterns associated with drowsiness. The approach

uses geometric relationships between vertical and horizontal eye distances to determine whether the eyes are open or closed. Due to its simplicity and low computational complexity, EAR-based detection became widely used in lightweight drowsiness monitoring systems.

Several studies also focused on physiological signal-based drowsiness detection systems using EEG, EOG, and heart rate monitoring sensors. These systems provide highly accurate fatigue detection by analyzing brain activity and eye movement signals. However, such approaches require wearable devices and expensive hardware equipment, making them uncomfortable and difficult for practical real-world usage. Researchers identified the need for non-intrusive and low-cost monitoring systems capable of real-time deployment.

Google Research introduced MediaPipe Face Mesh as a lightweight and efficient framework for real-time facial landmark detection. MediaPipe Face Mesh can identify 468 facial landmarks accurately and supports real-time processing on CPU-based systems. Many recent drowsiness detection systems use MediaPipe Face Mesh for extracting eye coordinates and facial features because of its fast processing speed and high detection accuracy. The framework significantly improved real-time performance in webcam-based driver monitoring applications.

Deep Learning approaches using Convolutional Neural Networks (CNN) and Long Short-Term Memory (LSTM) networks were also explored for drowsiness detection. CNN models analyze facial images and eye states, while LSTM models identify temporal behavioral patterns over continuous video frames. These approaches improve prediction accuracy and support

multi-state classification. However, Deep Learning methods require large training datasets, GPU hardware, and high computational power, making lightweight deployment difficult on standard systems.

Recent research combined Computer Vision techniques with lightweight Machine Learning algorithms such as Logistic Regression for real-time drowsiness monitoring. These systems use webcam frames, facial landmark extraction, and EAR calculation to classify driver states efficiently. Logistic Regression models provide fast prediction speed and low computational complexity while maintaining stable real-time performance. Such lightweight approaches are highly suitable for practical deployment on standard hardware systems without expensive computational resources.

Overall, previous studies demonstrated that vision-based drowsiness detection systems provide an effective and non-intrusive solution for driver monitoring. However, challenges such as lighting variations, face occlusion, head movement, and computational complexity still affect system performance. The proposed Driver Drowsiness Detection System addresses these limitations by integrating MediaPipe Face Mesh, Eye Aspect Ratio calculation, Logistic Regression classification, and real-time alert generation into a lightweight and efficient monitoring framework.

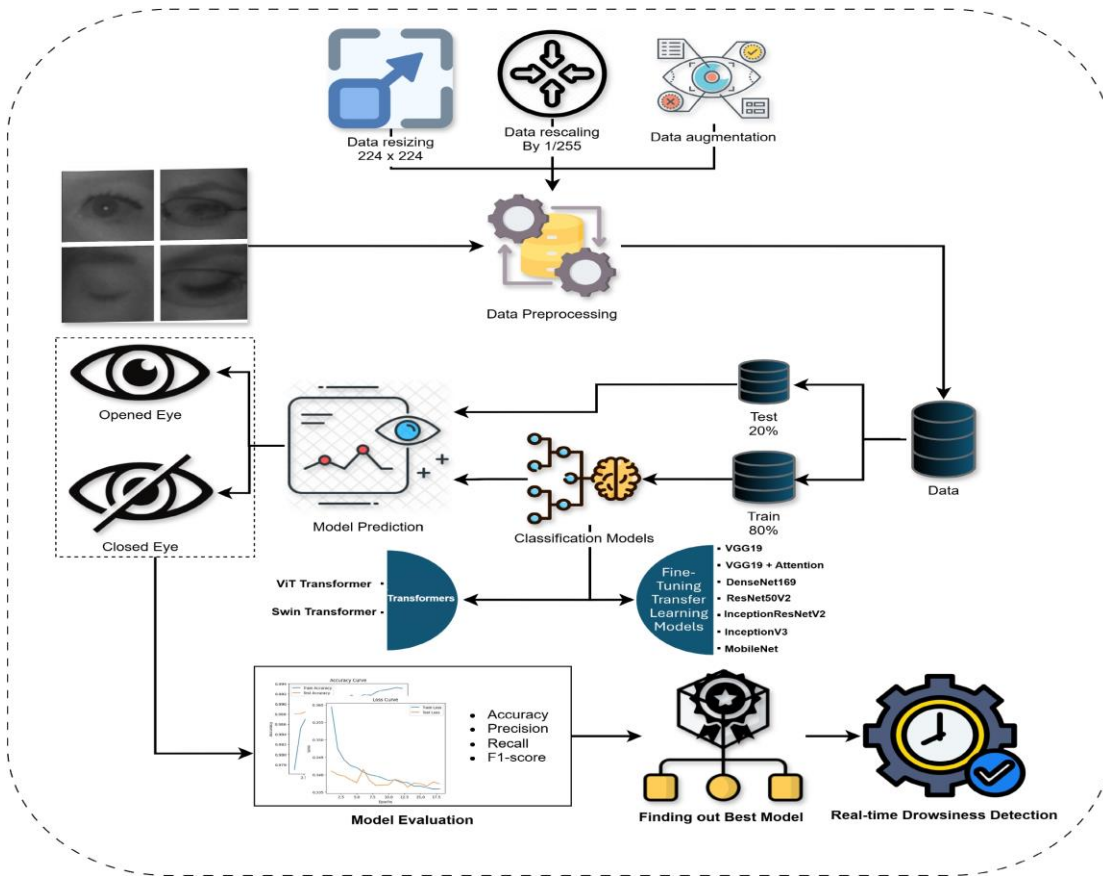
The proposed Driver Drowsiness Detection System continuously captures webcam frames and processes them using MediaPipe Face Mesh. Eye landmarks are extracted and Eye Aspect Ratio values are calculated. Logistic Regression classifies the driver state as Alert or Drowsy. Consecutive drowsy detections activate an audio alert using Web Audio API. The

frontend dashboard displays prediction results, confidence scores, webcam feed, and detection history dynamically.

3.1 SYSTEM ARCHITECTURE

The system architecture consists of frontend, backend, and Machine Learning modules. The frontend captures webcam frames and displays results. The backend processes image frames using OpenCV and MediaPipe Face Mesh. The Logistic Regression model performs prediction and returns results through Flask REST APIs. The modular architecture improves scalability and deployment efficiency.

The proposed Driver Drowsiness Detection System follows a lightweight and efficient architecture designed for real-time monitoring of driver alertness using Computer Vision and Machine Learning techniques. The architecture consists of multiple modules that work together to capture webcam frames, process facial landmarks, calculate Eye Aspect Ratio (EAR), predict driver state, and generate warning alerts. The system ensures smooth real-time performance with low computational complexity and supports deployment on standard hardware systems.



The proposed Driver Drowsiness Detection System follows a lightweight and efficient architecture designed for real-time monitoring of driver alertness using Computer Vision and Machine Learning techniques. The system consists of multiple modules that work together to capture webcam frames, process facial features, calculate Eye Aspect Ratio (EAR), predict driver state, and generate warning alerts in real time.

3.2.1 FRONTEND LAYER

The frontend layer is responsible for user interaction and dashboard visualization. It is developed using HTML, CSS, and JavaScript. The browser accesses the webcam through JavaScript APIs and continuously

captures video frames of the driver. The frontend dashboard displays live webcam feed, EAR values, confidence percentages, driver status, and alert notifications dynamically without refreshing the page.

The frontend also manages audio alerts using the Web Audio API whenever continuous drowsiness is detected. Dynamic updates improve user interaction and provide smooth real-time monitoring.

3.2.2 VIDEO FRAME CAPTURE MODULE

The video frame capture module continuously captures webcam frames from the browser. JavaScript camera APIs such as `getUserMedia` are used to access the webcam and obtain live video streams.

Frames are extracted at regular intervals using the Canvas API and converted into image format before being sent to the backend server for processing. Continuous frame capture helps the system monitor driver behavior in real time.

3.2.3 PROCESSING LAYER

The processing layer handles image analysis, facial landmark extraction, and Eye Aspect Ratio calculation. Webcam frames received from the frontend are processed using OpenCV and MediaPipe Face Mesh.

MediaPipe Face Mesh identifies 468 facial landmark points accurately in real time. Eye landmark coordinates are extracted from these facial points for further analysis. The processing layer ensures efficient and low-latency facial feature extraction.

3.2.4 FACIAL LANDMARK DETECTION MODULE

The facial landmark detection module uses MediaPipe Face Mesh to detect facial structures and identify important eye regions. The framework extracts facial landmark coordinates from webcam frames with high accuracy.

The eye landmarks are selected from the detected facial points, and geometric relationships between these coordinates are used for Eye Aspect Ratio calculation. Accurate landmark extraction is essential for reliable drowsiness detection.

3.2.5 EYE ASPECT RATIO CALCULATION MODULE

The Eye Aspect Ratio (EAR) calculation module measures eye openness using vertical and horizontal eye distances. EAR values remain high when the eyes are open and decrease significantly during blinking or eye closure.

Continuous monitoring of EAR values helps identify prolonged eye closure patterns associated with driver drowsiness. EAR acts as the primary feature used for Machine Learning prediction.

3.2.6 MACHINE LEARNING PREDICTION MODULE

The Machine Learning module uses Logistic Regression for classifying the driver state into Alert or Drowsy categories. The model is trained using synthetic EAR datasets representing different eye conditions.

The calculated EAR values are passed to the Logistic Regression model, which generates prediction results along with confidence percentages. The

lightweight model ensures fast prediction speed with low computational complexity.

3.2.7 BACKEND LAYER

The backend layer is implemented using Python Flask and handles communication between frontend and processing modules. The Flask backend receives webcam frames from the frontend through REST API communication.

The backend processes image frames, calculates EAR values, performs Machine Learning prediction, and returns prediction results to the frontend dashboard in JSON format. The backend ensures efficient real-time processing and low response time.

3.2.8 ALERT GENERATION MODULE

The alert generation module continuously monitors consecutive drowsy predictions. If the system detects prolonged eye closure for multiple frames, an audio alert is activated immediately to warn the driver.

The Web Audio API is used to generate alert sounds dynamically. The system also supports cooldown and mute features to avoid repetitive notifications and improve user experience.

3.2.9 REAL-TIME DASHBOARD MODULE

The dashboard module displays all monitoring results dynamically in real time. The dashboard includes live webcam feed, EAR values, prediction confidence, driver status, and detection logs.

The interactive interface improves visualization and helps users understand the monitoring process clearly. Real-time updates are implemented using JavaScript without requiring page refresh.

3.2.10 SYSTEM PERFORMANCE AND SCALABILITY

The proposed system architecture is lightweight, modular, and scalable. Since the system uses MediaPipe Face Mesh and Logistic Regression instead of computationally expensive Deep Learning models, it can run efficiently on standard CPU-based systems without GPU acceleration.

The modular design also allows future integration of advanced features such as CNN and LSTM models, yawning detection, head pose estimation, and cloud-based monitoring systems. Overall, the architecture ensures efficient real-time performance, low latency, and reliable drowsiness detection suitable for practical transportation safety applications.

3.2 HARDWARE REQUIREMENTS

- Intel i3 Processor or above
- Minimum 4 GB RAM
- Webcam

- 500 GB Storage
- Laptop/Desktop System

3.3 SOFTWARE REQUIREMENTS

- Python
- Flask
- OpenCV
- MediaPipe
- HTML
- CSS
- JavaScript
- VS Code

4. MODULE DESCRIPTION

The system contains modules for webcam capture, facial landmark extraction, EAR calculation, Machine Learning prediction, dashboard visualization, and alert generation. Each module performs a specific task and contributes to the overall workflow of real-time drowsiness detection.

The proposed Driver Drowsiness Detection System consists of multiple functional modules that work together to perform real-time driver monitoring and drowsiness detection. Each module performs a specific operation such as webcam frame capture, facial landmark extraction, Eye Aspect Ratio calculation, Machine Learning prediction, alert generation, and dashboard visualization. The modular architecture improves system efficiency, scalability, and real-time performance.

4.1 VIDEO FRAME CAPTURE MODULE

The video frame capture module is responsible for accessing the webcam and continuously capturing live video frames of the driver. The frontend application uses JavaScript APIs such as `getUserMedia` to establish webcam access through the browser.

Captured frames are extracted at regular intervals using the Canvas API and converted into image format for backend processing. Continuous frame capture enables real-time monitoring of driver eye movements and facial behavior.

This module acts as the input source for the entire system and ensures smooth video streaming with minimal delay.

4.2 FACIAL LANDMARK DETECTION MODULE

The facial landmark detection module uses MediaPipe Face Mesh for identifying facial structures and extracting landmark coordinates from webcam frames. MediaPipe Face Mesh detects 468 facial landmark points accurately in real time.

The module identifies important eye landmarks required for Eye Aspect Ratio calculation. Accurate facial landmark detection is essential because the system's prediction accuracy depends heavily on precise eye coordinate extraction.

The lightweight design of MediaPipe Face Mesh supports fast processing and low computational complexity on standard CPU-based systems.

4.3 EYE ASPECT RATIO CALCULATION MODULE

The Eye Aspect Ratio (EAR) calculation module computes EAR values using eye landmark coordinates extracted from MediaPipe Face Mesh. EAR is calculated using the geometric relationship between vertical and horizontal eye distances.

When the driver's eyes remain open, EAR values stay relatively high. During blinking or eye closure, EAR values decrease significantly. Continuous monitoring of EAR values helps identify drowsiness patterns associated with prolonged eye closure.

The calculated EAR value acts as the primary feature used for Machine Learning classification.

4.4 MACHINE LEARNING PREDICTION MODULE

The Machine Learning prediction module uses Logistic Regression to classify the driver state as Alert or Drowsy. The model is trained using synthetic EAR datasets representing different eye conditions.

The module receives EAR values from the preprocessing stage and predicts the driver's condition in real time. Along with prediction results, the model also generates confidence percentages indicating prediction reliability.

Logistic Regression is selected because of its lightweight nature, fast prediction speed, and low computational requirements, making it suitable for real-time applications.

4.5 BACKEND PROCESSING MODULE

The backend processing module is implemented using Python Flask. It manages communication between frontend and Machine Learning modules using REST API calls.

The backend receives webcam frames from the frontend, performs image preprocessing, extracts facial landmarks, calculates EAR values, and executes Machine Learning prediction. The prediction results are returned to the frontend dashboard in JSON format.

This module ensures smooth data flow and efficient real-time communication between all system components.

4.6 ALERT GENERATION MODULE

The alert generation module continuously monitors drowsiness predictions and activates warning alerts whenever prolonged eye closure is detected.

If consecutive drowsy predictions exceed the predefined threshold, the module generates an audio alert using the Web Audio API. The alert mechanism helps notify drivers immediately and improves road safety by reducing accidents caused by fatigue.

The module also supports cooldown periods and mute controls to improve usability and avoid repetitive alerts.

4.7 DASHBOARD VISUALIZATION MODULE

The dashboard visualization module provides a user-friendly frontend interface for displaying monitoring results dynamically in real time. The dashboard is developed using HTML, CSS, and JavaScript.

The interface displays live webcam feed, EAR values, confidence percentages, prediction status, and detection logs continuously. Dynamic updates are implemented without refreshing the page, improving system responsiveness and user interaction.

The visualization module helps users monitor system performance clearly and understand driver alertness conditions effectively.

4.8 DATA LOGGING MODULE

The data logging module stores and updates prediction results, EAR values, confidence scores, and drowsiness events continuously during monitoring.

Detection logs help analyze driver behavior patterns and system performance over time. Statistical information such as total scans, drowsy events, and alert counts can also be maintained for performance evaluation.

The logging mechanism improves monitoring reliability and supports future analysis and system optimization.

4.9 CONTINUOUS MONITORING MODULE

The continuous monitoring module ensures uninterrupted real-time monitoring of driver behavior and alertness conditions. The module

continuously processes webcam frames and updates prediction results dynamically.

A streak-based detection mechanism is implemented to reduce false alarms caused by normal blinking. The system triggers alerts only when multiple consecutive drowsy predictions occur.

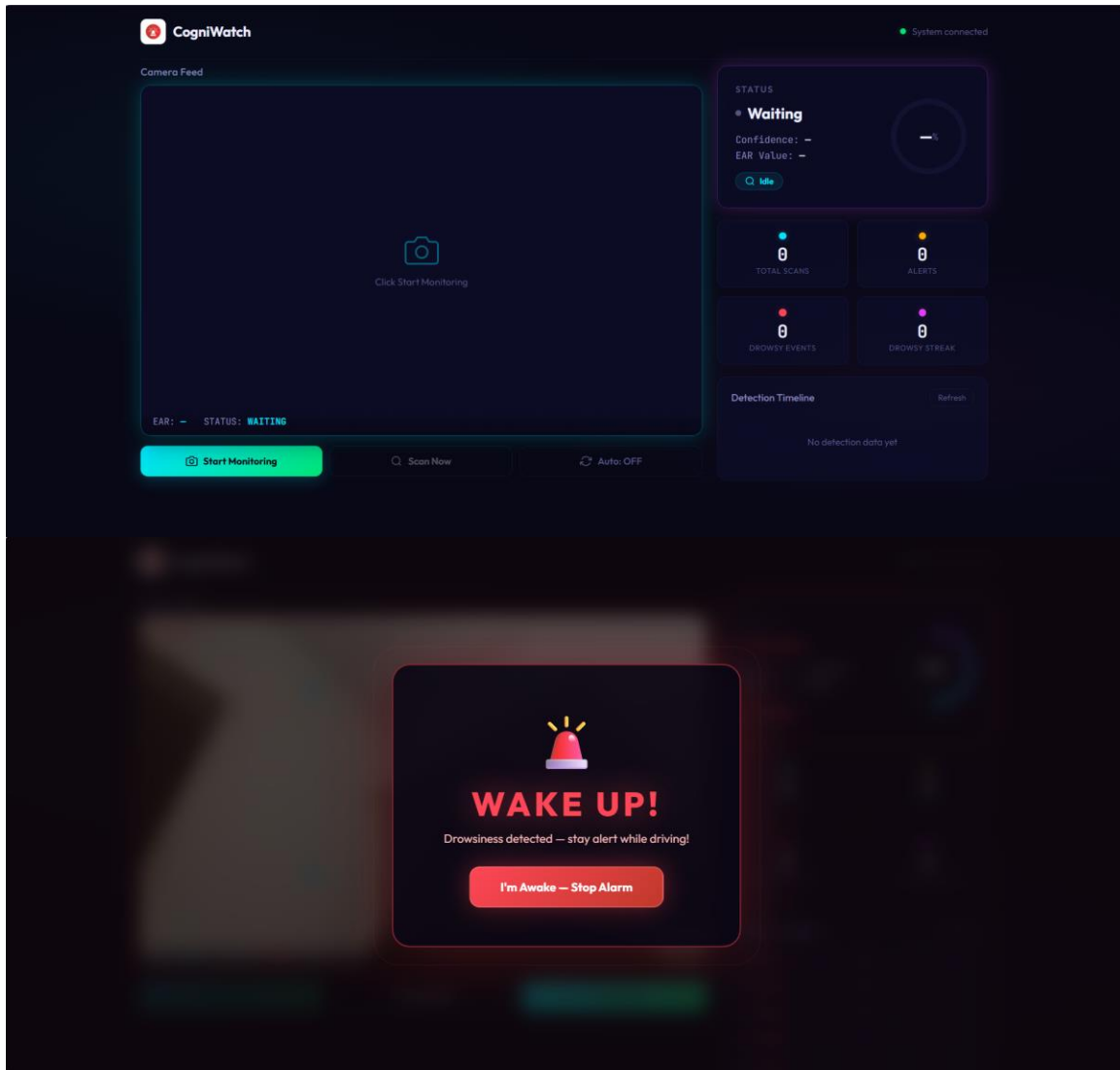
Continuous monitoring improves system reliability and provides stable real-time performance for practical driver safety applications.

Top of Form

Bottom of Form

5. IMPLEMENTATION AND RESULTS

The proposed system was implemented successfully using Flask backend and JavaScript frontend dashboard. MediaPipe Face Mesh extracted facial landmarks accurately and Logistic Regression generated stable predictions. The frontend dynamically displayed webcam feed, EAR values, confidence percentages, and detection logs. The system produced efficient real-time performance under normal testing conditions.



The proposed Driver Drowsiness Detection System was implemented successfully using Computer Vision and Machine Learning techniques for real-time monitoring of driver alertness. The system integrates frontend technologies, backend processing, facial landmark extraction, Eye Aspect Ratio calculation, Machine Learning prediction, and alert generation into a lightweight real-time application. Experimental testing demonstrated efficient performance, low latency, and stable prediction accuracy under normal operating conditions.

5.1 SYSTEM IMPLEMENTATION

The system implementation involves integrating webcam-based video processing, MediaPipe Face Mesh, Logistic Regression, Flask backend, and frontend dashboard modules into a single real-time monitoring application.

The frontend interface was developed using HTML, CSS, and JavaScript for webcam access and dashboard visualization. Python Flask was used for backend communication and Machine Learning integration. OpenCV and MediaPipe Face Mesh handled facial landmark extraction and image processing tasks.

The complete implementation supports real-time processing without requiring GPU hardware, making the system suitable for deployment on standard CPU-based systems.

5.2 FRONTEND IMPLEMENTATION

The frontend dashboard was implemented using HTML, CSS, and JavaScript to provide an interactive and user-friendly monitoring interface. The browser accesses the webcam using JavaScript APIs such as `getUserMedia`.

Live webcam frames are captured continuously and displayed dynamically on the dashboard. The interface also displays Eye Aspect Ratio values, confidence percentages, detection status, alert notifications, and detection logs in real time.

JavaScript is used for sending webcam frames to the backend through REST API calls and updating prediction results dynamically without refreshing the page.

5.3 BACKEND IMPLEMENTATION

The backend implementation was developed using Python Flask. The Flask server manages communication between frontend and Machine Learning modules through REST APIs.

The backend receives webcam frames from the frontend, decodes image data, processes frames using OpenCV and MediaPipe Face Mesh, calculates Eye Aspect Ratio values, and performs Machine Learning prediction using Logistic Regression.

Prediction results are returned to the frontend in JSON format along with confidence percentages and driver status. The backend ensures efficient real-time communication and low processing delay.

5.4 FACIAL LANDMARK EXTRACTION RESULTS

MediaPipe Face Mesh successfully extracted 468 facial landmark points from webcam frames in real time. The framework accurately detected eye regions under normal lighting conditions and provided stable landmark extraction performance.

Eye coordinates were selected from the facial landmarks for Eye Aspect Ratio calculation. The system demonstrated reliable facial landmark

detection for different users and maintained consistent performance during continuous monitoring.

However, slight variations in performance were observed under poor lighting conditions, partial face occlusion, and extreme head movements.

5.5 EYE ASPECT RATIO CALCULATION RESULTS

The Eye Aspect Ratio calculation module successfully identified eye openness and closure patterns associated with drowsiness. EAR values remained relatively high when the eyes were open and decreased significantly during blinking or prolonged eye closure.

Continuous monitoring of EAR values helped identify drowsiness patterns accurately. The system successfully differentiated between normal blinking and prolonged eye closure using threshold-based analysis and streak detection mechanisms.

The EAR-based approach provided lightweight and efficient real-time performance with minimal computational complexity.

5.6 MACHINE LEARNING PREDICTION RESULTS

The Logistic Regression model was trained using synthetic EAR datasets representing Alert and Drowsy conditions. The model produced stable prediction results with low computational cost and fast response time.

Real-time EAR values were processed continuously by the model to classify the driver state as Alert or Drowsy. Prediction results were displayed dynamically on the dashboard along with confidence percentages.

The lightweight Logistic Regression model provided smooth real-time performance without requiring GPU acceleration or large computational resources.

5.7 ALERT GENERATION RESULTS

The alert generation module successfully activated audio warnings whenever prolonged drowsiness was detected continuously. The system used streak-based prediction logic to reduce false alarms caused by normal blinking.

When consecutive drowsy predictions exceeded the threshold limit, the Web Audio API generated alert sounds immediately to notify the driver. The alert mechanism improved reliability and ensured timely warning generation during fatigue conditions.

Cooldown and mute features also improved usability by preventing repetitive alert notifications.

5.8 REAL-TIME DASHBOARD RESULTS

The frontend dashboard displayed all monitoring results dynamically in real time. The dashboard included live webcam feed, EAR values, confidence percentages, driver status indicators, and detection logs.

Dynamic frontend updates improved user interaction and provided smooth visualization without requiring page refresh. The dashboard interface remained responsive during continuous monitoring and real-time prediction.

The visualization module helped users understand system behavior and monitor alertness conditions effectively.

5.9 SYSTEM PERFORMANCE ANALYSIS

The proposed system demonstrated efficient real-time performance with low computational requirements. MediaPipe Face Mesh and Logistic Regression provided lightweight processing suitable for CPU-based systems.

The system maintained low latency during webcam frame processing and prediction tasks. Consecutive drowsiness detection improved stability and reduced false alarms caused by temporary blinking.

Performance analysis showed that the lightweight architecture provided smooth monitoring and stable prediction accuracy for practical transportation safety applications.

6. CONCLUSION

The proposed “Real-Time Driver Drowsiness Detection System Using Eye Aspect Ratio and Lightweight Machine Learning” successfully demonstrates an efficient and lightweight solution for monitoring driver alertness in real time. The system continuously captures webcam video frames, detects facial landmarks using MediaPipe Face Mesh, calculates Eye Aspect Ratio (EAR) values, and classifies the driver state as Alert or Drowsy using Logistic Regression. The integration of Computer Vision and Machine Learning techniques enables accurate real-time drowsiness detection with low computational complexity.

The system effectively generates audio warning alerts whenever prolonged eye closure or fatigue conditions are detected continuously. The frontend dashboard dynamically displays live webcam feed, EAR values, confidence percentages, detection logs, and driver status, improving user interaction and monitoring efficiency. The lightweight architecture allows the application to run smoothly on standard CPU-based systems without requiring expensive sensors or GPU hardware.

Experimental results demonstrated stable real-time performance, low latency, and reliable detection accuracy under normal operating conditions. Although certain limitations such as poor lighting conditions and face occlusion affect performance slightly, the proposed system still provides a practical, low-cost, and scalable solution for intelligent driver monitoring applications. Overall, the project successfully contributes toward improving transportation safety by reducing accidents caused by driver fatigue and drowsiness through real-time monitoring and alert generation.

.